

Captura la bandera – documentación

Funcionalidad

Arquitectura del sistema

La solución implementa una arquitectura cliente-servidor basada en un servidor autoritativo. El sistema está compuesto por tres módulos principales: el servidor del juego, el Bridge y el cliente web. El servidor, desarrollado en Node.js, mantiene el estado oficial de la partida y aplica todas las reglas del protocolo. El Bridge funciona como intermediario entre el navegador y el servidor, traduciendo la comunicación WebSocket del navegador hacia conexiones TCP y UDP. Finalmente, el cliente, desarrollado con Vite y Three.js, representa la escena 3D y envía únicamente las acciones realizadas por el usuario.

Funcionamiento general

El juego es un Capture The Flag en 3D (con Three.js) que sigue el protocolo estándar de la clase: comunicación por TCP/UDP en crudo, mensajes en JSON separados por saltos de línea. El proyecto se divide en varias piezas que se hablan entre sí:

- Servidor del juego (Node, TCP + UDP): cumple el protocolo de la clase.
- Bridge (Node, WebSocket local): un "traductor" que corre en la máquina local.
- Cliente (navegador, Vite + Three.js): la parte visual, habla con el bridge por WebSocket local.

Flujo de conexión

Al iniciar la aplicación, el navegador establece una conexión WebSocket con el Bridge local. El Bridge envía mensajes de descubrimiento mediante UDP al broadcast general y al broadcast de las interfaces de red disponibles. Los servidores activos responden con su información (nombre, puerto TCP, estado y cantidad de jugadores), permitiendo que el cliente muestre la lista de servidores disponibles. Una vez seleccionado un servidor, el Bridge abre una conexión TCP y el cliente envía un mensaje join para registrarse en la partida. El servidor responde con los mensajes welcome y lobby, proporcionando la configuración del juego y la lista de jugadores conectados.

El ciclo de vida de una partida

- **lobby:** esperando jugadores. Necesita mínimo 2 para pasar a countdown.
- **countdown:** cuenta regresiva de 5 segundos. Al llegar a 0, se asigna a cada jugador una posición aleatoria en un anillo alrededor del centro (nunca dentro del círculo).
- **playing:** el servidor mueve a todos según su última dirección recibida, revisa si alguien ganó, y resuelve los "interact" pendientes (en el orden en que llegaron, sin excepciones).
- **finished:** se anuncia el ganador, se espera 5 segundos, y se vuelve solo al lobby.

La bandera

La bandera vive en el centro del mapa cuando está libre. Capturarla o robarla es lo mismo: basta con estar a 40 unidades de distancia. Lo importante es la condición de victoria: no basta con estar afuera del círculo, hay que haber hecho la transición real.

El formato de los mensajes

Todo mensaje es un objeto JSON de una sola línea, terminado en un salto de línea (\n). Como TCP entrega los bytes en un chorro continuo (sin separar mensajes), protocol/parser.js va acumulando lo que llega en un buffer, y cada vez que encuentra un \n, corta ahí: eso es un mensaje completo, lo parsea, y guarda el resto para la siguiente vuelta.

Los mensajes principales: join (unirse), input (hacia dónde me muevo), interact (tomar/robar bandera), welcome/lobby/countdown/start/state/game_over (los manda el servidor), y error (cuando algo salió mal, con un código tipo LOBBY_FULL o GAME_STARTED).

Bridge (medida tomada para poder usar el navegador)

El Bridge permite utilizar un navegador web en un proyecto basado en TCP y UDP. Mantiene una conexión WebSocket con el cliente y administra internamente el descubrimiento de servidores, las conexiones TCP y el reenvío de mensajes entre ambos extremos. Además, implementa el mecanismo de conexión manual por dirección IP como respaldo cuando el descubrimiento automático no está disponible.

Cliente

- **Network.js:** habla con el bridge por WebSocket. No sabe nada de TCP, solo manda/recibe JSON.
- **main.js:** Escucha los mensajes del servidor y decide qué mostrar (menú, cuenta regresiva, escena 3D, pantalla de victoria).
- **SceneManager.js:** arma la escena de Three.js (mapa, círculo, luces) y coloca a los jugadores y la bandera según las coordenadas que manda el servidor.
- **Player.js:** cada jugador es una esfera con su nombre flotando encima. El jugador local es verde; los demás, gris oscuro.
- **inputHandler.js:** traduce WASD/flechas a un vector (-1, 0 o 1 en cada eje) y el espacio/E a "quiero interactuar". El servidor es quien normaliza el movimiento diagonal, el cliente solo manda la intención cruda.

Modo espectador (anfitrión)

Quien corre el servidor en su máquina solo debe poder ver la partida, no jugarla. Un espectador se registra para recibir todos los mensajes (así puede ver la partida en vivo), pero el servidor nunca lo agrega a la lista de jugadores: no cuenta para el mínimo de 2, no aparece en el estado del juego, y si intenta mandar input o interact, el servidor se lo rechaza. El navegador tampoco le crea controles de teclado cuando entra en este modo.

Manejo de errores

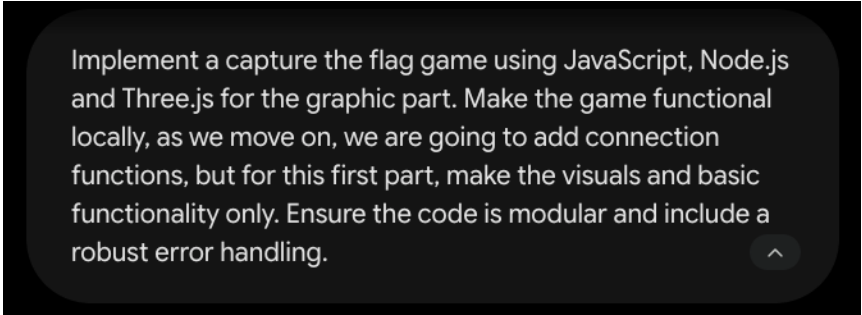
El servidor valida todos los mensajes recibidos antes de modificar el estado de la partida. Si un mensaje contiene datos inválidos, se recibe fuera de la fase correspondiente o incumple alguna regla del protocolo, el servidor responde con un mensaje error indicando el motivo mediante códigos como INVALID_PHASE, GAME_STARTED, LOBBY_FULL, NAME_INVALID o INVALID_FIELD. De esta forma se evita que acciones incorrectas afecten la sincronización de la partida

Historial de Commits

Para la implementación del proyecto se inició utilizando Google Gemini, para experimentar, así que, la base del proyecto está hecha con ayuda de Google Gemini. También se utilizó Claude y ChatGPT para la corrección de algunos errores. En Github se encuentran todos los commits realizados durante el tiempo de desarrollo.

Commit 1

Se desarrolló la estructura inicial del proyecto utilizando JavaScript, Node.js y Three.js. En esta etapa se construyó una primera versión funcional del juego ejecutándose de forma local, priorizando una arquitectura modular que permitiera agregar posteriormente la comunicación en red. También se establecieron las bases para separar la lógica del juego de la parte gráfica y se incorporó un manejo básico de errores para facilitar el desarrollo futuro. (Gemini)

A screenshot of a chat interface with a dark background. The text is white and reads: "Implement a capture the flag game using JavaScript, Node.js and Three.js for the graphic part. Make the game functional locally, as we move on, we are going to add connection functions, but for this first part, make the visuals and basic functionality only. Ensure the code is modular and include a robust error handling." There is a small upward arrow icon at the bottom right of the text area.

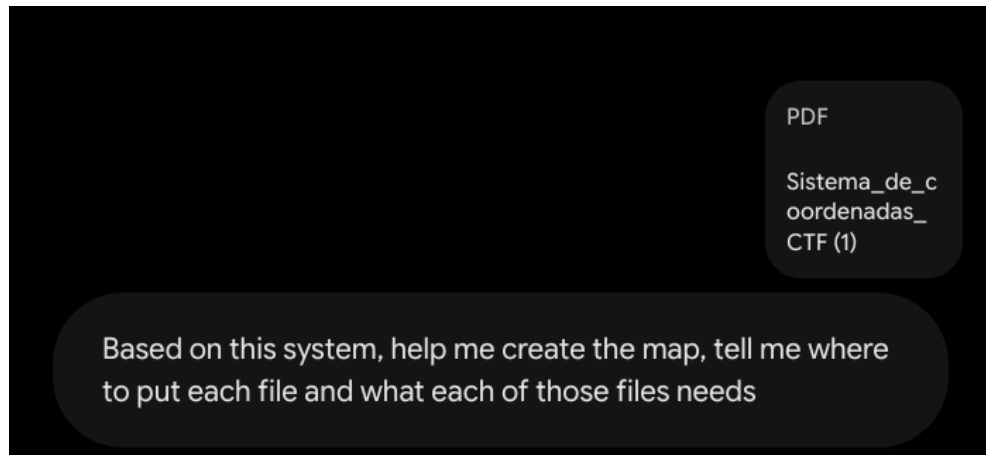
Implement a capture the flag game using JavaScript, Node.js and Three.js for the graphic part. Make the game functional locally, as we move on, we are going to add connection functions, but for this first part, make the visuals and basic functionality only. Ensure the code is modular and include a robust error handling.

Commit 2 update of structure

Se reorganizó completamente la estructura del proyecto para separar las responsabilidades de cada componente. Se dividió el código en módulos independientes para el servidor, el cliente, el bridge y el manejo del protocolo de comunicación. Esta reorganización permitió que el proyecto fuera más escalable y facilitó la incorporación de nuevas funcionalidades sin afectar el resto del sistema.

Commit 3

Con base en el protocolo proporcionado para el laboratorio, se definió la organización general del proyecto y la responsabilidad de cada archivo. Además, se construyó la primera versión del mapa del juego, estableciendo el sistema de coordenadas, el área central donde se ubica la bandera y la distribución general de los elementos que posteriormente utilizaría el servidor para validar el movimiento de los jugadores. (Gemini)



Commit 4 visuals

Se implementó la lógica necesaria para restringir el movimiento dentro del mapa, controlar las colisiones con los límites, verificar cuándo un jugador puede interactuar con la bandera y detectar las condiciones de victoria.

Commit 5 Initial version of discovery

Se implementó el sistema de descubrimiento automático de servidores mediante UDP Broadcast. Los clientes pueden enviar un mensaje de descubrimiento a la red local y recibir como respuesta la información de los servidores disponibles. Además, se incorporó un mecanismo de conexión manual mediante dirección IP para permitir la conexión incluso cuando el broadcast se encuentra bloqueado por la red. (Gemini)

Implement the Client Side UDP discovery, it has to do the following:
Create a UDP socket to broadcast a discovery message {"type": "discover", "v": 1} to the network broadcast address (255.255.255.255) on port 8888.
Listen for incoming responses (server_info) which contain the server's name, TCP port, state, and player count.
Provide a manual IP input fallback option in case network broadcast is blocked by a router or Wi-Fi.

Commit 6

Se creó la clase GameState, responsable de almacenar toda la información relacionada con la partida. Esta clase administra la lista de jugadores conectados, las fases del juego, la posición y propietario de la bandera, así como las reglas necesarias para determinar cuándo un jugador gana la partida. (Gemini)

Let's implement state.js, it needs to do the following:

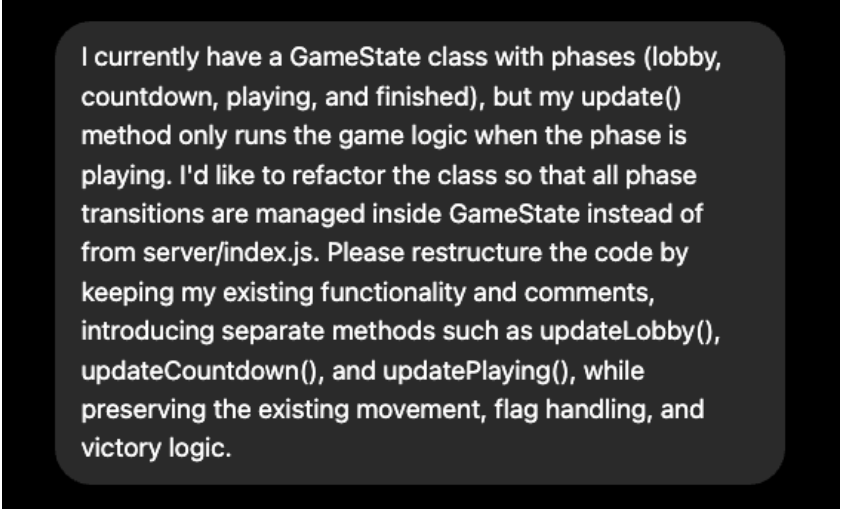
- Tracks the 1000 x 1000 map boundaries and the central circle radius (300)
- Stores active players, their IDs, names, and clamped positions (between 15 and 985 to respect player radius)
- Tracks whether the flag is free (at center 500,500) or owned, and processes movement updates and victory conditions when a carrier exits the central circle

Commit 7

PENDIENTE

Commit 8

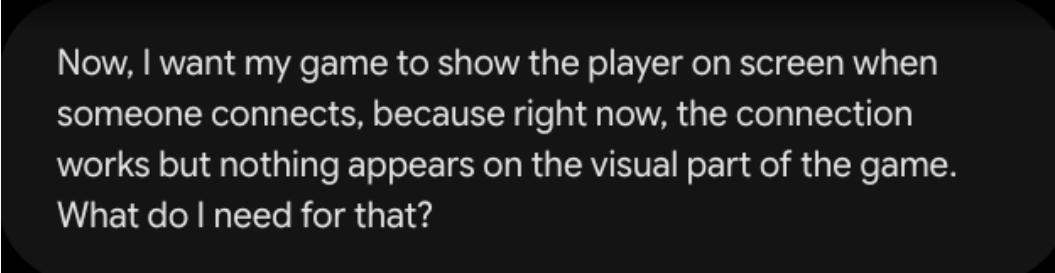
Se reorganizó el funcionamiento interno de la clase GameState para que cada fase del juego fuera administrada mediante métodos independientes. Se separó el comportamiento correspondiente al lobby, la cuenta regresiva, el desarrollo de la partida y el reinicio del juego, logrando un código mucho más limpio, mantenible y fácil de extender. (ChatGPT)

A screenshot of a chat message from ChatGPT, displayed in a dark-themed interface. The message is contained within a rounded rectangular bubble. The text describes a request to refactor a GameState class by separating phase-specific logic into individual methods.

I currently have a GameState class with phases (lobby, countdown, playing, and finished), but my update() method only runs the game logic when the phase is playing. I'd like to refactor the class so that all phase transitions are managed inside GameState instead of from server/index.js. Please restructure the code by keeping my existing functionality and comments, introducing separate methods such as updateLobby(), updateCountdown(), and updatePlaying(), while preserving the existing movement, flag handling, and victory logic.

Commit 9 (sin querer no se subió este commit por separado)

Se implementó el mecanismo para representar visualmente a todos los jugadores conectados dentro de la escena de Three.js. A partir de los estados enviados por el servidor, el cliente crea, actualiza y elimina automáticamente los jugadores visibles en pantalla, manteniendo sincronizada la representación gráfica con el estado real del servidor. (Gemini)

A screenshot of a chat message from Gemini, displayed in a dark-themed interface. The message is contained within a rounded rectangular bubble. The text asks for guidance on how to visually represent players on the screen based on server data.

Now, I want my game to show the player on screen when someone connects, because right now, the connection works but nothing appears on the visual part of the game. What do I need for that?

Commit 10

Se desarrolló la representación tridimensional de la bandera dentro de la escena del juego. La bandera permanece inicialmente en el centro del mapa y, cuando un jugador la captura, comienza a desplazarse junto con él hasta que se cumpla la condición de victoria o sea reiniciada por el servidor. (Gemini)

Help me create the visual part of the flag for the game

To render the flag in your Three.js frontend, you need to create a 3D mesh that represents the flag (such as a pole with a colored banner or a glowing marker) and update its 3D position in your render loop whenever your client receives the latest `state` snapshot from the server.

Here is how you can set up the flag in your Three.js scene:

1. Create and Add the Flag to Your Scene

Add this code where you set up your Three.js scene, camera, and lights:

Commit 11

Se modificó el mecanismo mediante el cual el servidor distribuye la información del juego hacia todos los clientes. Se implementó un modelo basado en eventos que facilita la sincronización del estado, reduce la duplicación de código y mejora la organización interna del servidor.

Commit 12

Se fortaleció el módulo encargado de administrar las conexiones TCP entre el servidor y los clientes. Se agregaron validaciones adicionales para los mensajes recibidos, se optimizó el envío de estados a todos los jugadores conectados y se mejoró el manejo de errores durante la comunicación.

Commit 13

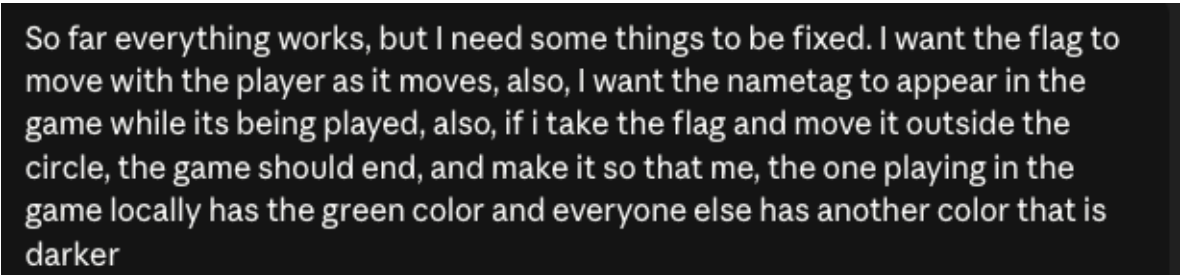
Se incorporó al SceneManager la capacidad de crear, actualizar y eliminar jugadores automáticamente según la información recibida desde el servidor. Esto permitió que la escena gráfica reflejara correctamente las conexiones y desconexiones sin necesidad de reiniciar el cliente.

Commit 14

Se logró completar el flujo completo de comunicación entre el navegador, el bridge y el servidor. A partir de este punto el navegador fue capaz de descubrir servidores disponibles, establecer conexiones correctamente y recibir los estados enviados por el servidor para actualizar la escena de juego.

Commit 15

Se realizaron mejoras generales sobre la arquitectura de comunicación para mantener separada la lógica de red de la representación gráfica. También se optimizó la sincronización entre el estado autoritativo del servidor y la representación visual mostrada en el navegador. (Claude)



So far everything works, but I need some things to be fixed. I want the flag to move with the player as it moves, also, I want the nametag to appear in the game while its being played, also, if i take the flag and move it outside the circle, the game should end, and make it so that me, the one playing in the game locally has the green color and everyone else has another color that is darker

Commit 16

Se desarrolló una nueva interfaz de usuario para facilitar la conexión al juego. Se incorporó un menú moderno con descubrimiento automático de servidores, conexión manual mediante dirección IP, lista de jugadores presentes en el lobby y visualización de la cuenta regresiva antes del inicio de la partida. (Claude)

I would like to add a nicer looking menu to join the game, so that it looks more intuitive and if I am the host, I can see the players joining and I can choose when to start the game. Please work first on the frontend part, then, the connctions.

Commit 17

Se incorporaron al proyecto las modificaciones desarrolladas por otros integrantes del grupo. Fue necesario revisar la compatibilidad entre ambas versiones, adaptar la arquitectura existente y resolver conflictos para mantener el cumplimiento del protocolo definido para el laboratorio. (Claude)

My classmates did some updates, here is the file that they sent with some new stuff plus my current files. Please implement the updates needed.

Commit 18

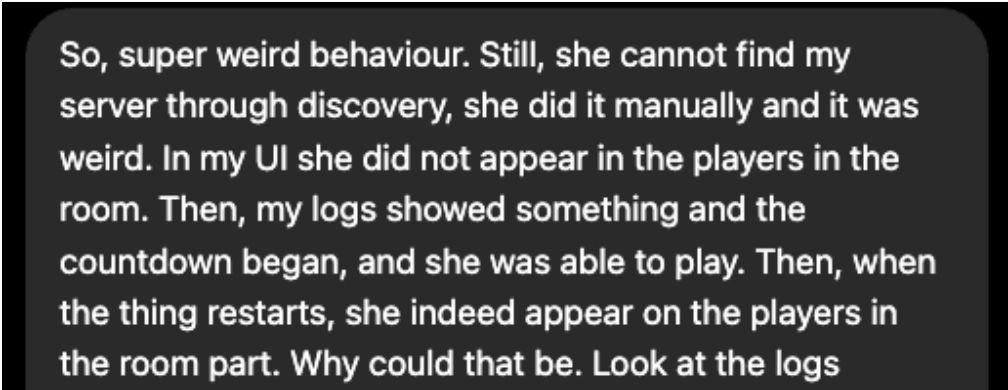
Se agregó la posibilidad de ejecutar el cliente en modo espectador para el anfitrión del servidor. En este modo el usuario puede observar la partida en tiempo real, pero no participa como jugador, no puede moverse ni interactuar con la bandera y tampoco cuenta para el mínimo de jugadores requerido para iniciar la partida.

Commit 19

Se solucionaron diversos problemas detectados durante las pruebas del juego, incluyendo el mapeo de controles mediante WASD, validaciones adicionales para las conexiones, mejoras en el descubrimiento manual de servidores y correcciones relacionadas con la estabilidad general del sistema.

Commit 20

Se investigaron y corrigieron problemas relacionados con el descubrimiento de servidores y la sincronización del lobby entre distintos equipos de la red. Se realizaron mejoras en el proceso de descubrimiento periódico, la normalización de identificadores de jugadores y la actualización de la interfaz para garantizar que los jugadores aparezcan correctamente en la sala antes del inicio de la partida. (ChatGPT)



So, super weird behaviour. Still, she cannot find my server through discovery, she did it manually and it was weird. In my UI she did not appear in the players in the room. Then, my logs showed something and the countdown began, and she was able to play. Then, when the thing restarts, she indeed appear on the players in the room part. Why could that be. Look at the logs

Commit 21

Se realizó una limpieza general del proyecto eliminando comentarios, fragmentos de código obsoletos y elementos que ya no eran necesarios después de las refactorizaciones realizadas durante el desarrollo.